Fault Tolerant Systems - Overview

(Fault Tolerance in Distributed Systems)

2110473 Fault Tolerant Computing

Contents

- Introduction
- Terminology of Fault Tolerance
- Failure Rate and Reliability
- Reliability and MTTF, MTTR
- Availability
- Strategies to handle Faults
- Redundancy Techniques
- Practice Cases in Fault Tolerance
  - RAID, TMR, Space Shuttle, S/W Faults, Fault Tolerance in Process Group, Reliable Client-Server Communication, Checkpointing & Recovery

Introduction

- Fault Tolerance
  - Ability of system to continue Error-free operation in presence of unexpected fault
- Important in mission-critical applications
  - E.g., medical, aviation, banking, etc.
  - Errors very costly
- Becoming important in mainstream applications
  - Technology scaling causing circuit behavior to become less predictable and more trends to failures
  - Needing fault tolerance to keep Failure rate within acceptable levels

Concepts of Fault Tolerance

- Hardware, software and networks cannot be totally free from failures
- Fault tolerance is a non-functional (QoS) requirement that requires a system to continue to operate, even in the presence of faults
- Fault tolerance should be achieved with minimal involvement of users or system administrators
- Distributed systems can be more fault tolerant than centralized systems, but with more processor hosts generally the occurrence of individual faults is likely to be more frequent

Attributes, Consequences and Strategies

Attributes
- Availability
- Reliability
- Safety
- Confidentiality
- Integrity
- Maintainability

What is a Dependable system?

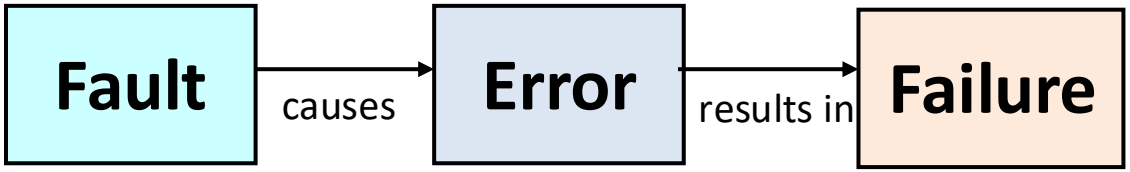
Consequences
- Fault
- Error
- Failure

How to distinguish faults?

Strategies
- Fault prevention
- Fault tolerance
- Fault recovery
- Fault forecasting

How to handle faults?

Terminology of Fault Tolerance



- **Fault** – is a defect within the system
- **Error** – is observed by a deviation from the expected behavior of the system
- **Failure** occurs when the system can no longer perform as required (does not meet spec)
- **Fault Tolerance** – is ability of system to provide a service, even in the presence of errors

Types of Fault (*wr*t time)

Permanent (Hard) Fault – repeatable error, e.g. failed component, Wearout failures, power fail, fire, flood, design error (usually software), sabotage

Soft Fault

Transient – occurs once or seldom, often due to unstable environment (e.g. bird flies past microwave transmitter)

Intermittent – occurs randomly, but where factors influencing fault are not clearly identified, e.g. unstable component

Operator error – human error

Types of Fault (*wrt* attributes)

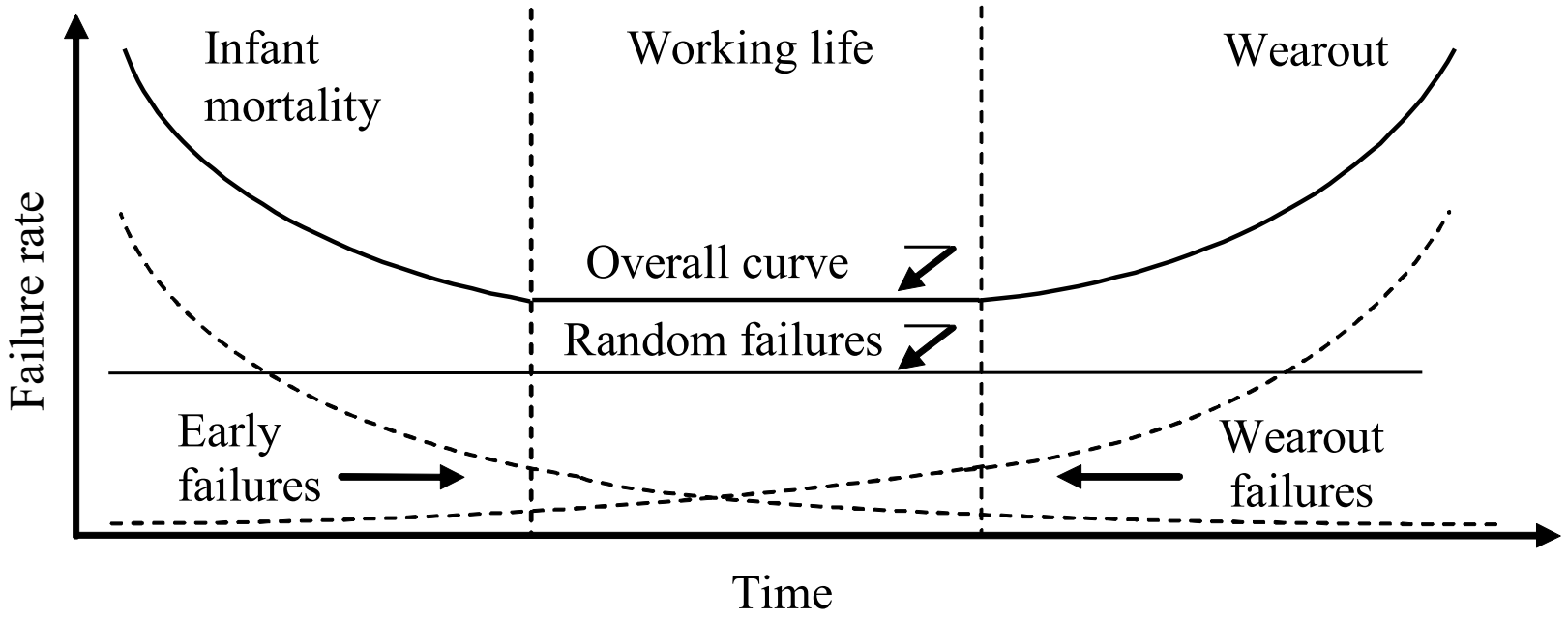
(Failure Mode of Distributed Systems)

Type of failure	Description
Crash failure	A server halts, but is working correctly until it halts
Amnesia crash	Lost all history, must be reboot
Pause crash	Still remember state before crash, can be recovered
Halting crash	Hardware failure, must be replaced or re-installed
Omission failure	A server fails to respond to incoming requests
Receive omission	A server fails to receive incoming messages
Send omission	A server fails to send messages
Timing failure	A server's response lies outside the specified time interval
Response failure	The server's response is incorrect
Value failure	The value of the response is wrong
State transition failure	The server deviates from the correct flow of control
Arbitrary failure	A server may produce arbitrary responses at arbitrary times

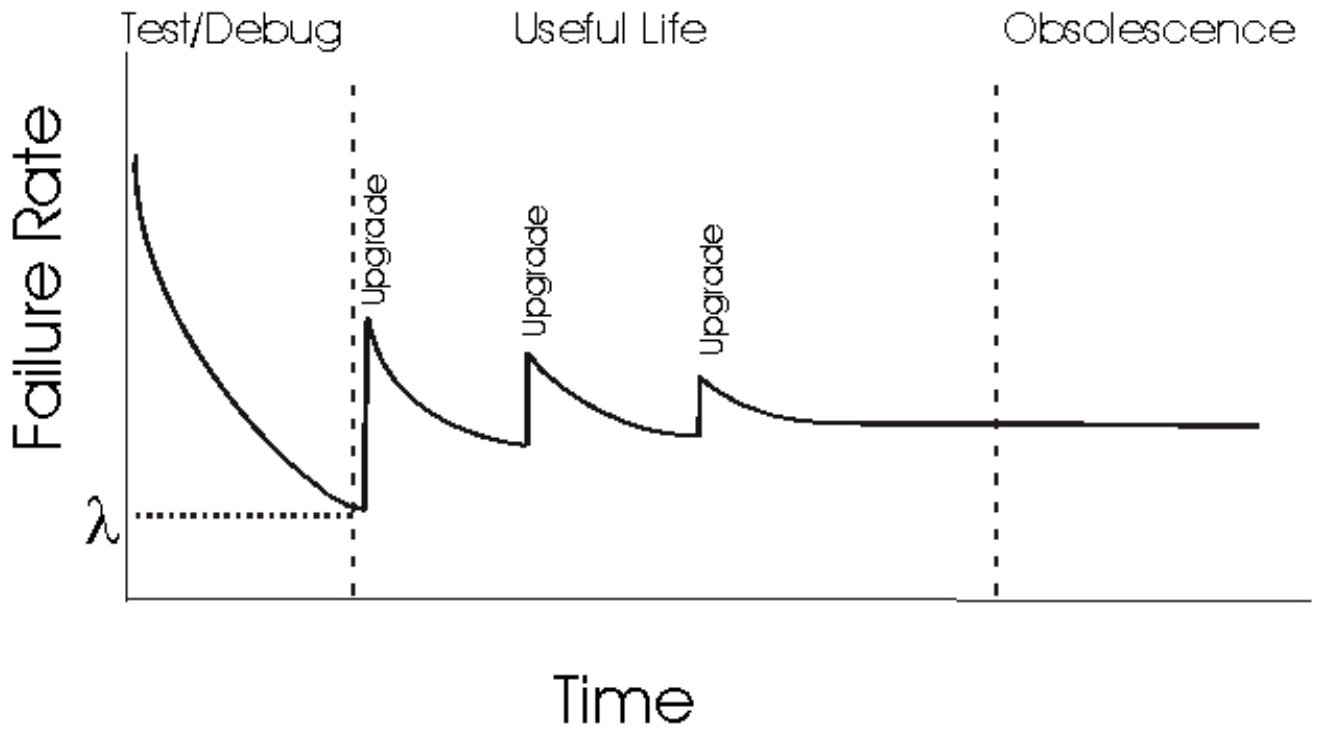
Failure Rate

- $\lambda(t)$ = Component failure rate
  - Measured in FITS (failures per 10^9 hours)

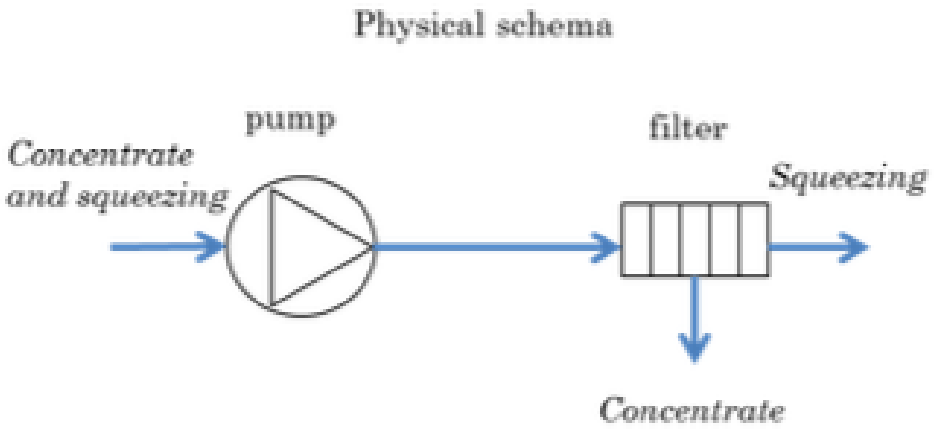
e.g. NASA Flight = 1 FIT, Computers = 10^5-10^6 FITs, Human Life (85yrs) = 1,343 FITs

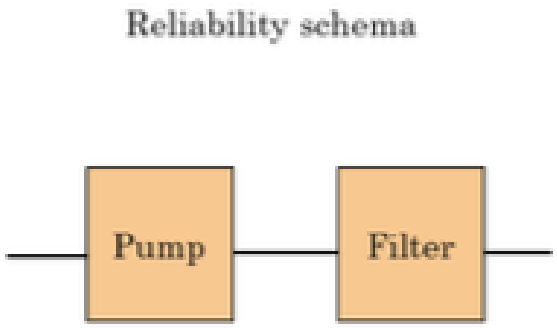


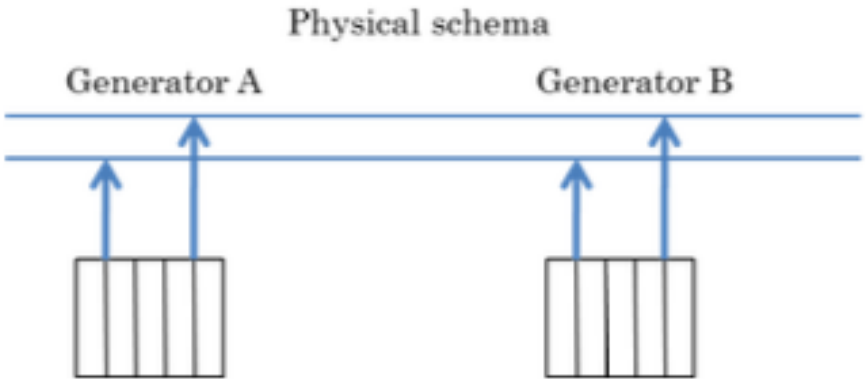
Bathtub curve for software reliability

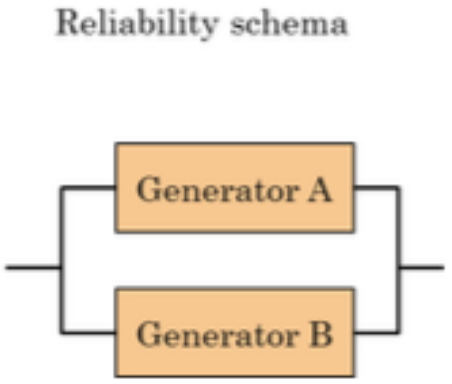


Reliability Modeling



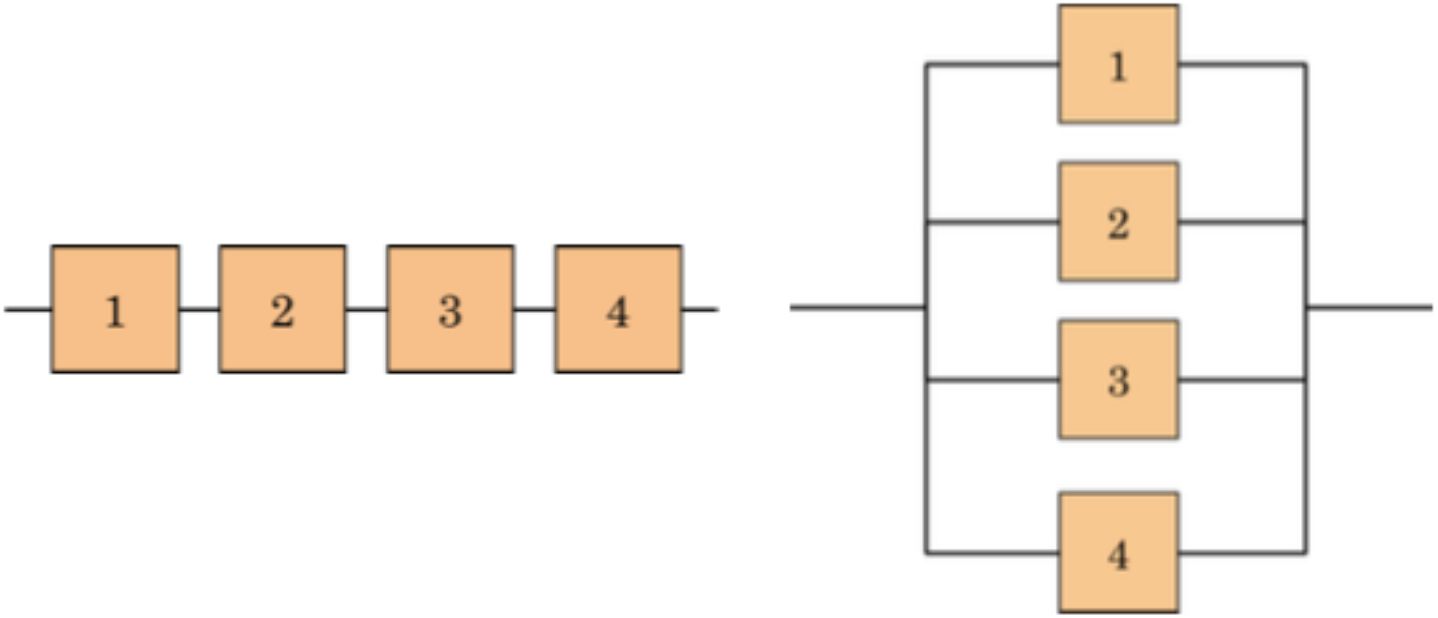




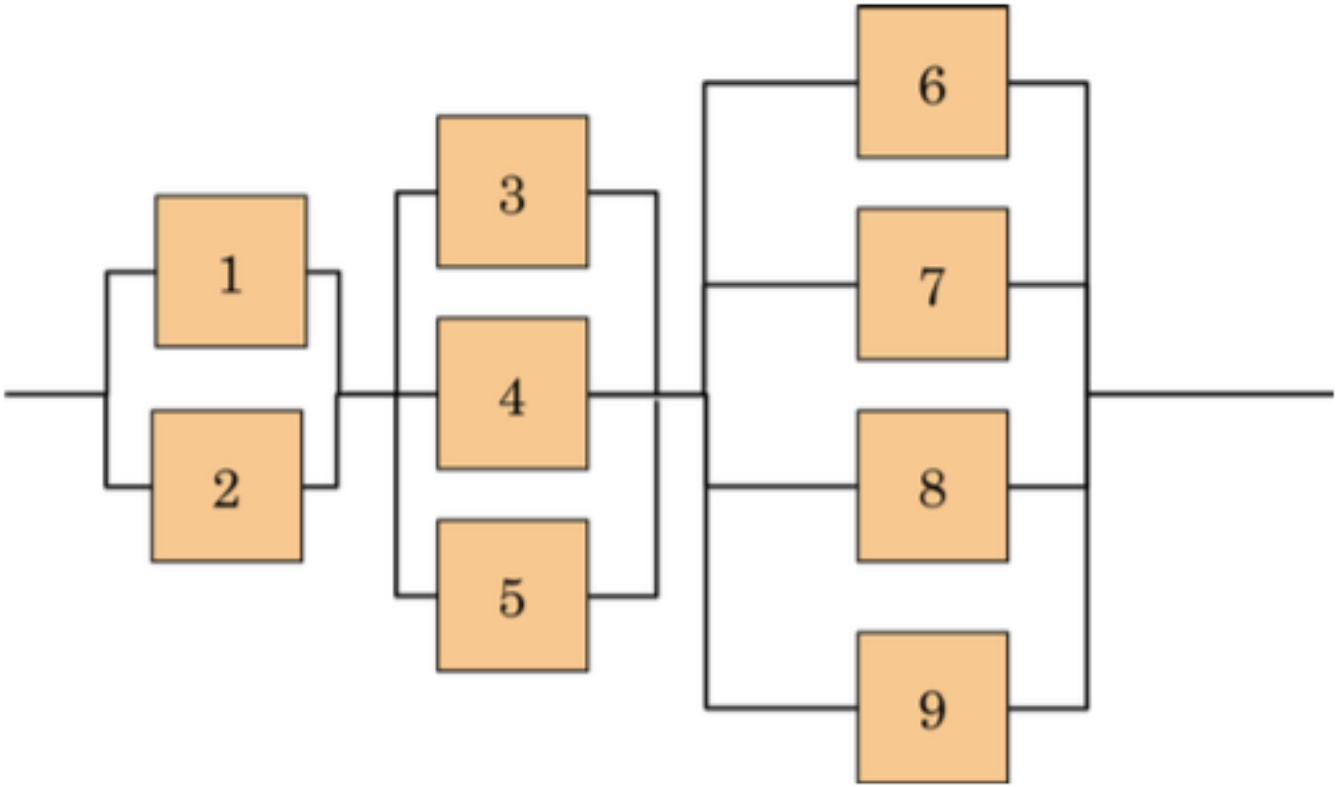


Reliability Block Diagram

- Series system works if all component work
- Parallel system works if @least one work

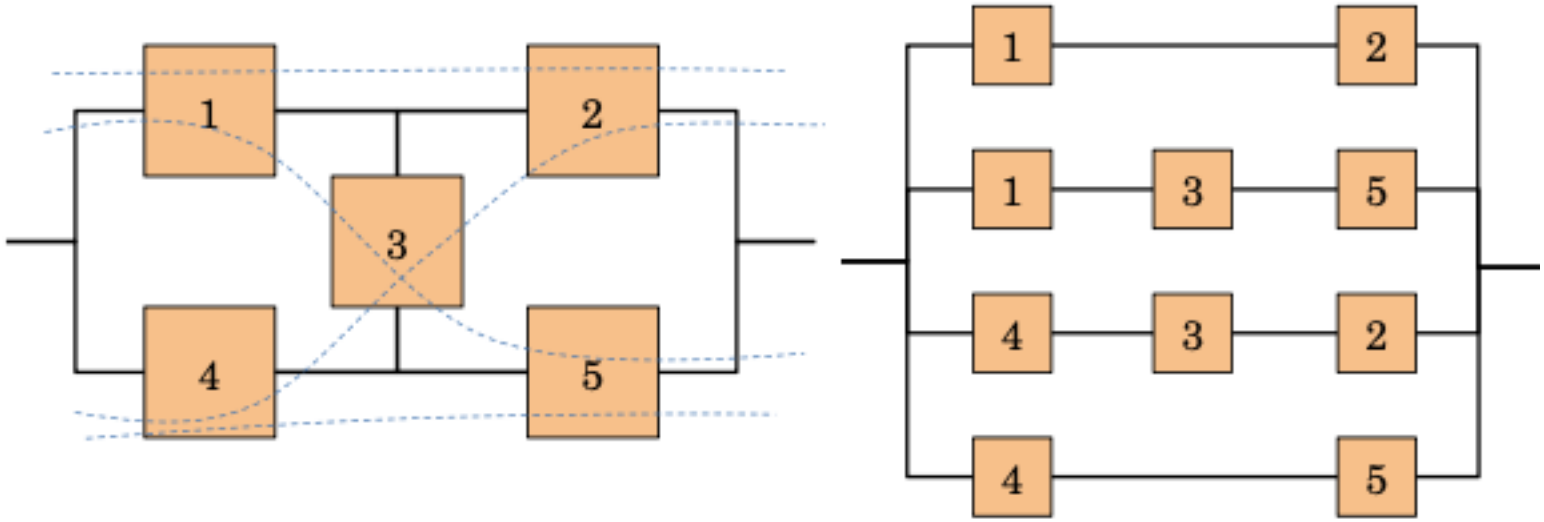


Series-Parallel System



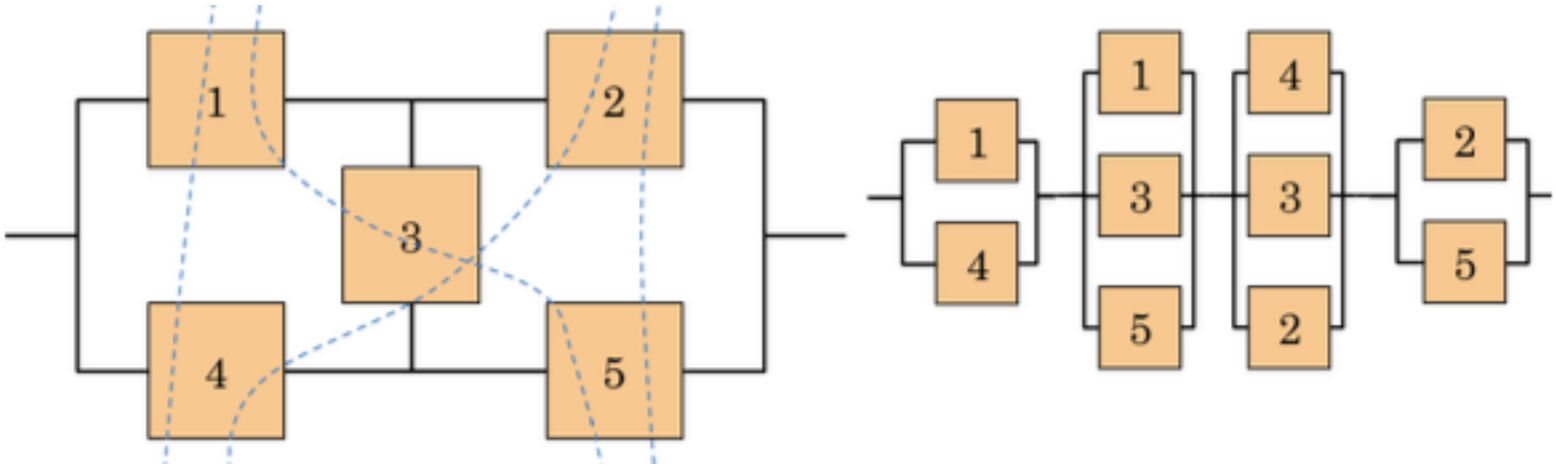
Minimal Path Set

- A subset of the system components such that the operation of all the components in the subset implies the operation of the system.



Minimal Cut Set

- A subset of the system components, such that the failure of all components in the subset does not imply the operation of the system



System Failure Rate

- System constructed from components
- No Fault Tolerance
  - Any component fails, whole system fails

$$\lambda_{sys} = \sum_{i=1}^{k} \lambda_{c,i}$$

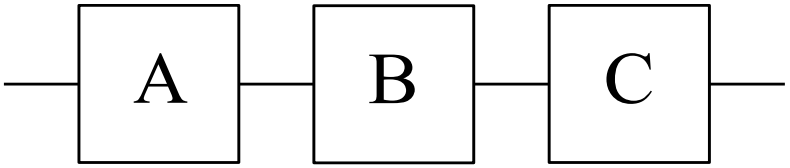
Reliability

- If component working at time 0
  - *R(t)* = Probability still working at time *t*
- Exponential Failure Law
  - If failure rate assumed constant
    - Good approximation if past infant mortality period

$$R(t) = e^{-\lambda t}$$

Reliability for Series System

- Series System
  - All components need to work for system to work



$$R_{sys} = R_A R_B R_C$$

How to make more Reliable

Reliability of *n*-Component system, each having failure rate λ:

$$R(t) = e^{-n\lambda t}$$

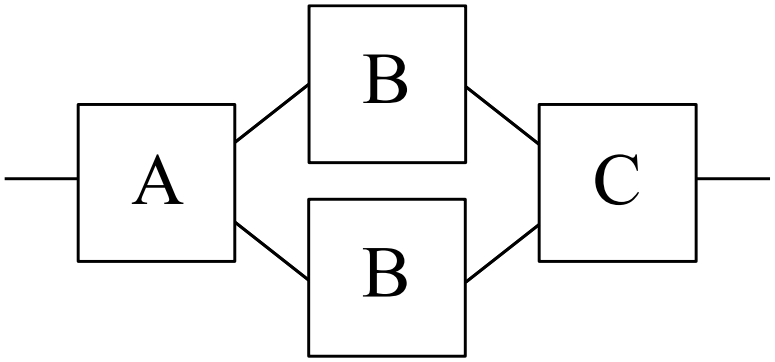
- *Reduce Failure Rate* λ
- *Reduce Number of Components n*
- *Reduce Time t*

Alternative:

- ***Introduce Redundancy***

System Reliability with Redundancy

- System reliability with component *B* in Parallel
  - Can tolerate one component *B* failing



$$R_{sys} = R_A\left[1-(1-R_B)^2\right]R_C = R_A(2R_B - R_B^2)R_C$$

System Reliability

- Series System

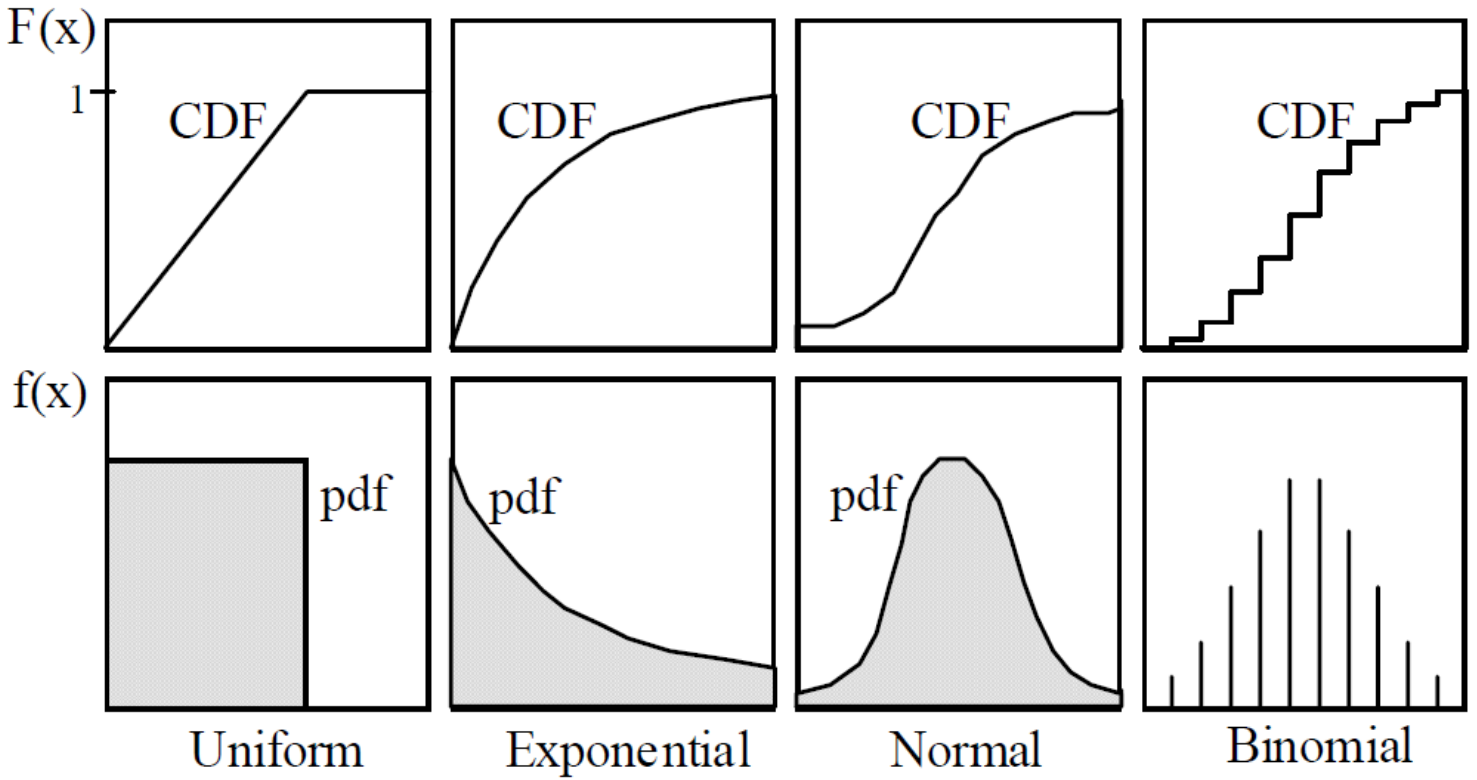
$$R_{overall} = \prod_{i=1}^{N} R_i$$

- Parallel System

$$R_{overall} = 1 - \prod_{i=1}^{N} (1 - R_i)$$

Some Simple Probability Distributions

- Cumulative distribution function: CDF $f(t) = \frac{prob[t \le x \le t+dt]}{dt} = dF(t)/dt$
- Probability density function: pdf $F(t) = prob[x \le f] = \int_0^t f(x)dx$



Reliability and MTTF

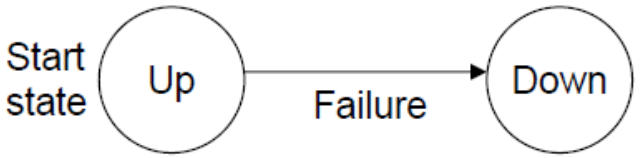
Reliability: *R*(*t*)

Probability that system remains in the "Good" state through the interval [0, *t*]

$R(t + dt) = R(t)\,[1 - z(t)\,dt]$

Hazard function

Two-state nonrepairable system



$R(t) = 1 - F(t)$ ------ CDF of the system lifetime, or its unreliability

Constant hazard function $z(t) = \lambda \Rightarrow R(t) = e^{-\lambda t}$
(system failure rate is independent of its age)

Exponential reliability law

Mean time to failure: MTTF

$$\text{MTTF} = \int_0^{+\infty} t\,f(t)\,dt = \int_0^{+\infty} R(t)\,dt$$

Expected value of lifetime

Area under the reliability curve (easily provable)

Mean-Time-to-Failure (MTTF)

- Average time before system fails
  - Equal to area under reliability curve

$$MTTF = \int_0^\infty R(t)dt$$

- For Exponential Failure Law

$$MTTF = \int_0^\infty e^{-\lambda t}dt = \frac{1}{\lambda}$$

Comparing Reliabilities

Reliability difference: $R_2 - R_1$

Reliability gain: R_2 / R_1

Reliability improvement factor

$RIF_{2/1} = [1-R_1(t_M)] / [1-R_2(t_M)]$

Example:

[1 – 0.9] / [1 – 0.99] = 10

Reliability improv. index

$RII = \log R_1(t_M) / \log R_2(t_M)$

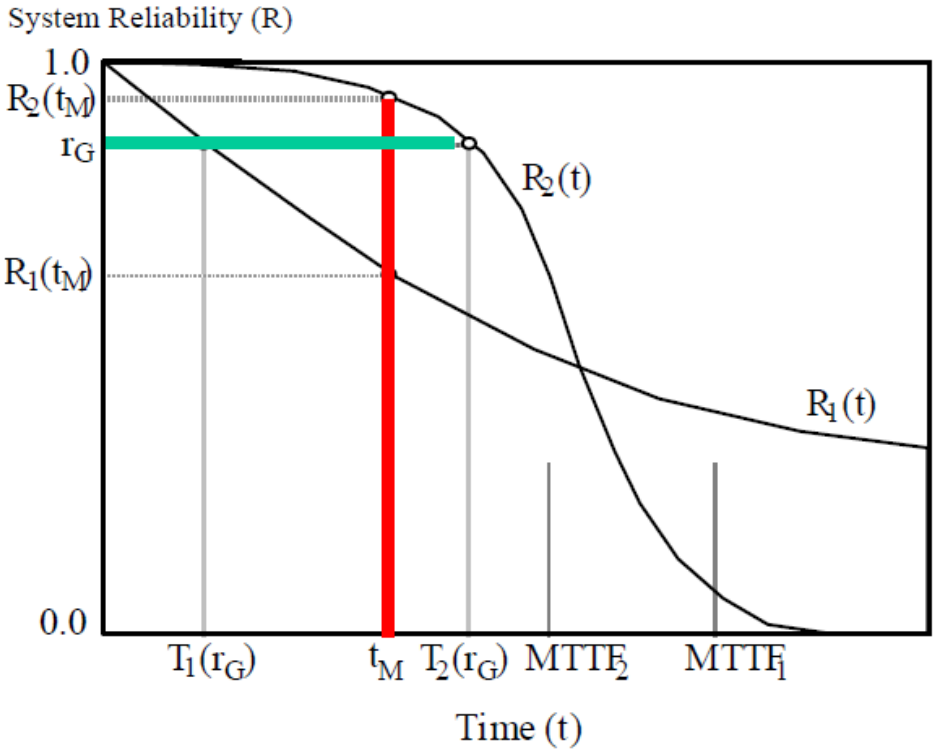
Mission time extension

$MTE_{2/1}(r_G) = T_2(r_G) - T_1(r_G)$

Mission time improv. factor:

$MTIF_{2/1}(r_G) = T_2(r_G) / T_1(r_G)$

Reliability functions for Systems 1/2



Repair Rate and MTTR

- μ = rate at which system repaired
  - constant for the exponential case
  - analogous to failure rate λ
- Maintainability often modeled as

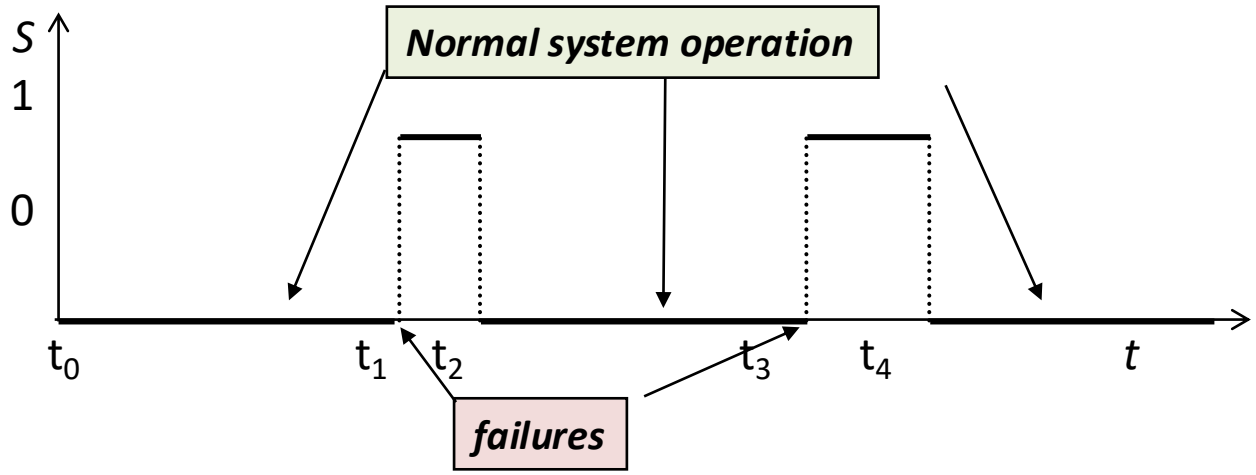
$$M(t) = 1 - e^{-\mu t}$$

- Mean-Time-to-Repair (MTTR) = $1/\mu$

Availability

- System Availability
  - Fraction of time system is operational

$$system\ availability = \frac{MTTF}{MTTF + MTTR}$$

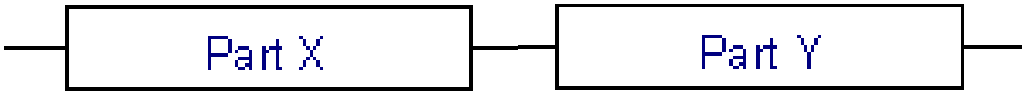


Availability (cont.)

- Telephone Systems
  - Required to have system availability of 0.9999 ("four nines")
- High-Reliability Systems
  - May require 7 or more nines
- Fault-Tolerant Design
  - Needed to achieve such high availability from less reliable components

Availability in Series

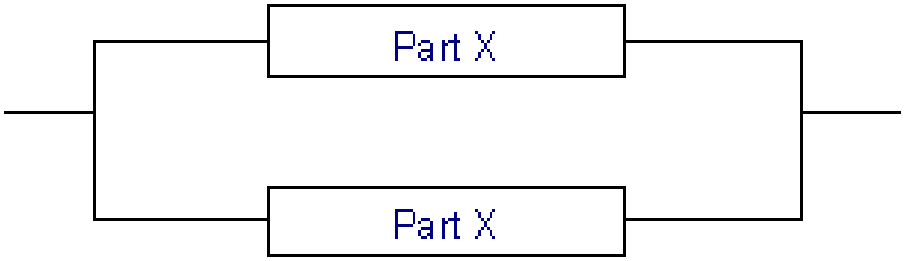
- $A = A_x A_y$



Component	Availability	Downtime
X	99% (2-nines)	3.65 days/year
Y	99.99% (4-nines)	52 minutes/year
X and Y Combined	98.99%	3.69 days/year

Availability in Parallel

- $A = 1-(1-A_x)^2$



Component	Availability	Downtime
X	99% (2-nines)	3.65 days/year
Two X components operating in parallel	99.99% (4-nines)	52 minutes/year
Three X components operating in parallel	99.9999% (6-nines)	31 seconds /year !

Simply Calculating Availability for Redundant Systems

- A is the probability that the system will be up (its availability)
- F is the probability that the system will be down
- a is the availability of a node
- n is the number of nodes in the system
- s is the number of spare nodes in the system
- f is the number of ways that all of the spares plus one other node can fail (that is, the number of node failures that will cause the system to fail)

Clearly, $A = 1 - F$

Simply, $A = 1 - f\,(1 - a)^{s+1}$

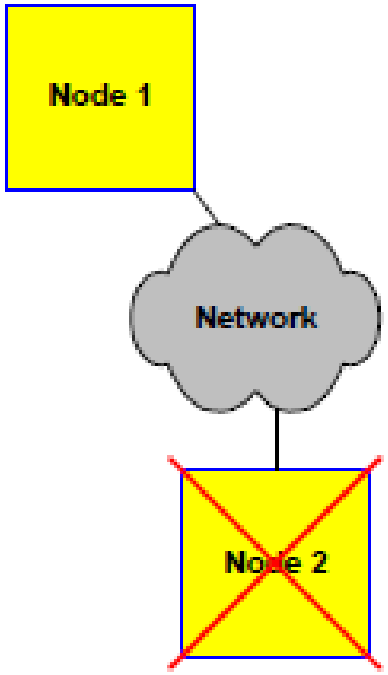
Dual Node Single Spare

- The probability *F* for the failure of the system

  $F = (1 - a)^2$

- The availability of the system, *A*, is

  $A = 1 - F = 1 - (1 - a)^2$



Note:
If the node availability is 0.99,
the probability of one node down is 0.01.
Thus, the both nodes down probability is $0.01^2 = 0.0001$
The system availability is 1 – 0.0001 = 0.9999 (four 9s)

Multiple Nodes Single Spare

- The number of Failure modes, *f*, is

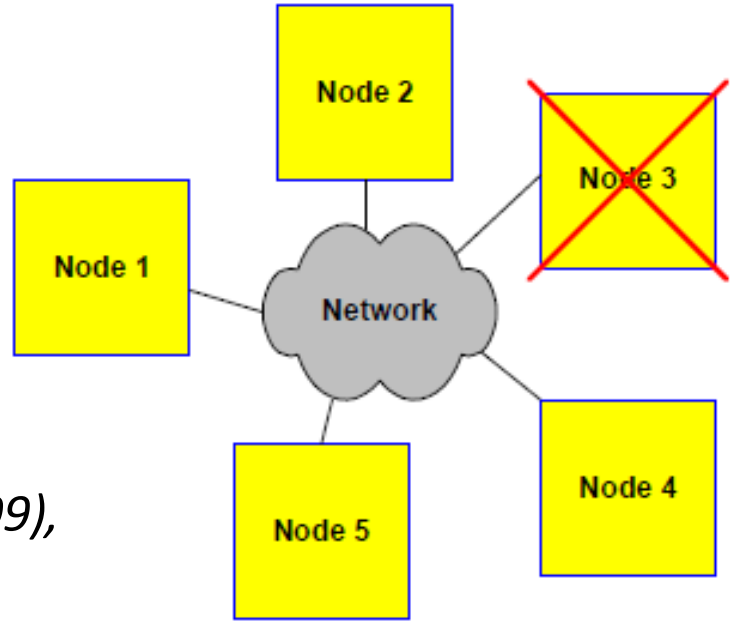
$$f = \frac{n(n-1)}{2}$$

- The probability of failure of the system, *F*, is the probability that any two nodes will fail

$$F = \frac{n(n-1)}{2}(1-a)^2$$

- System Availability, *A*, is

$$A = 1 - F = 1 - \frac{n(n-1)}{2}(1-a)^2$$



Note:
For Five-node system (node availability is 0.99),
The number of failure modes is ??
The availability of this five-node system is ??

Multiple Nodes Single Spare

- The number of Failure modes, *f*, is

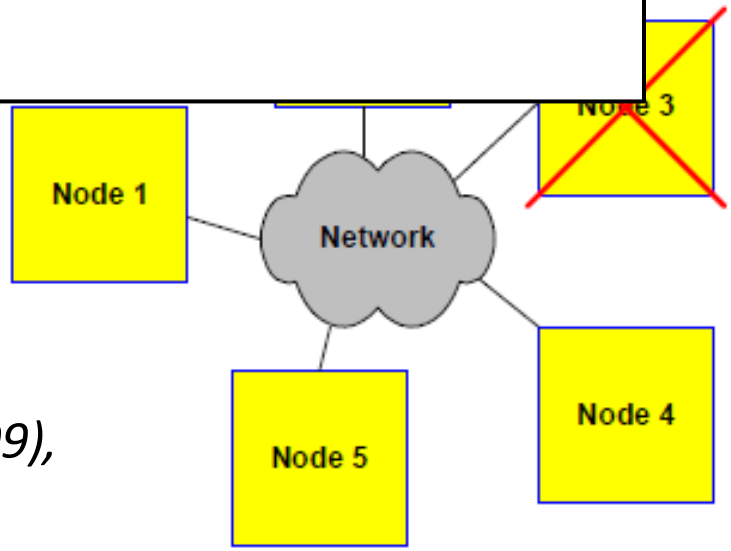
the number of failure modes is ten

- The probability of failure of the system, *F*, is the

the availability of a five-node system is [1 - 10(.01)2], or .999. This is three 9s of availability

- System Availability, *A*, is

$$A = 1 - F = 1 - \frac{n(n-1)}{2}(1-a)^2$$



Note:
For Five-node system (node availability is 0.99),
The number of failure modes is ??
The availability of this five-node system is ??

Multiple Nodes Multiple Spares

- The number of failure modes, *f*, is the number of ways that *s*+1 nodes out of *n* nodes can fail

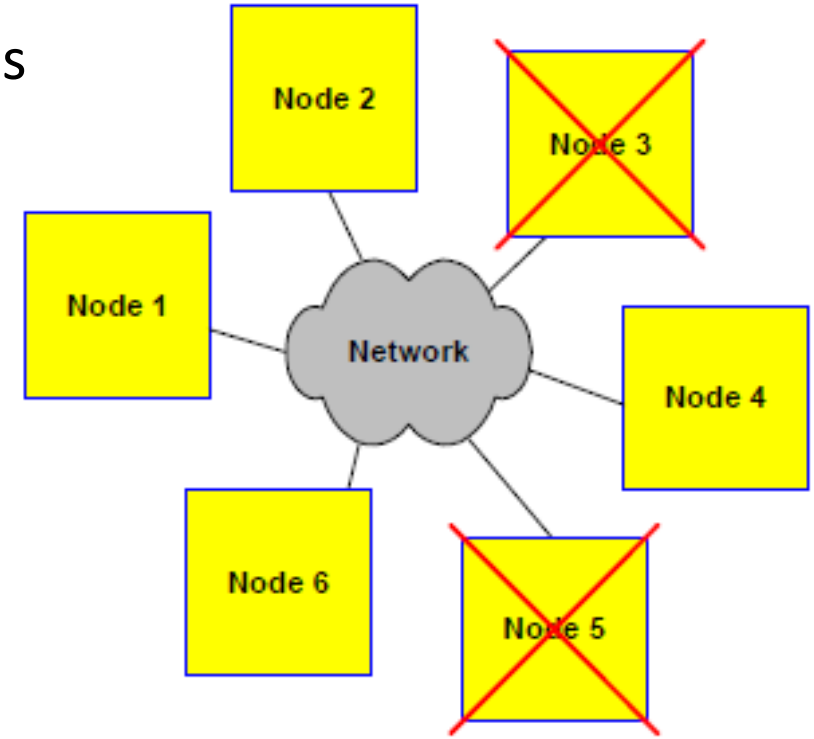
$$f = \frac{n!}{(s+1)!(n-s-1)!}$$

- The probability of failure is

$$F = f(1-a)^{s+1}$$

- The system availability is

$$A = 1 - f(1-a)^{s+1}$$



Note:
For Six-node system wt Two-spare (node availability is 0.99),
The number of failure modes is ?? => 20
The availability of this six-node system is ?? =>1-20(1-0.99)^3 = 0.99998*

Relationship Between Availability and Reliability

Reliability	Maintainability	Availability
Constant	Decrease	?
Constant	Increase	?
Increase	Constant	?
Decrease	Constant	?

Notes:

- *Reliability represents the probability of components, parts and systems to perform their required functions for a desired period of time without failure in specified environments with a desired confidence.*
- *Availability represents the probability that the system is capable of conducting its required function when it is called upon given that it is not failed or undergoing a repair action.*
- *An increase in maintainability implies a decrease in the time it takes to perform maintenance actions.*

- *Therefore, not only is availability a function of reliability, but it is also a function of maintainability.*

Difference Between Reliability and Availability

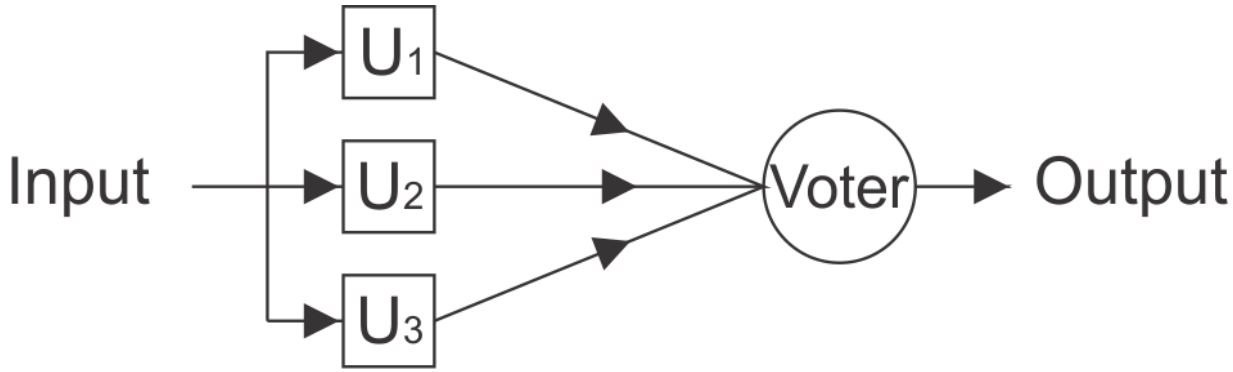
- Availability is a measure of the % of time the equipment is in an operable state
- While reliability is a measure of how long the item performs its intended function

- For example, *the machine is down 6 minutes every hour.*
  - This translates into an availability of 90% but a reliability of less than 1 hour
  - That may be okay in some circumstances but what if this is a paper machine?
  - It may take at least 30 minutes of run time to get to the point that we are producing good paper

- Generally speaking a reliable machine has high availability but an available machine may or may not be very reliable

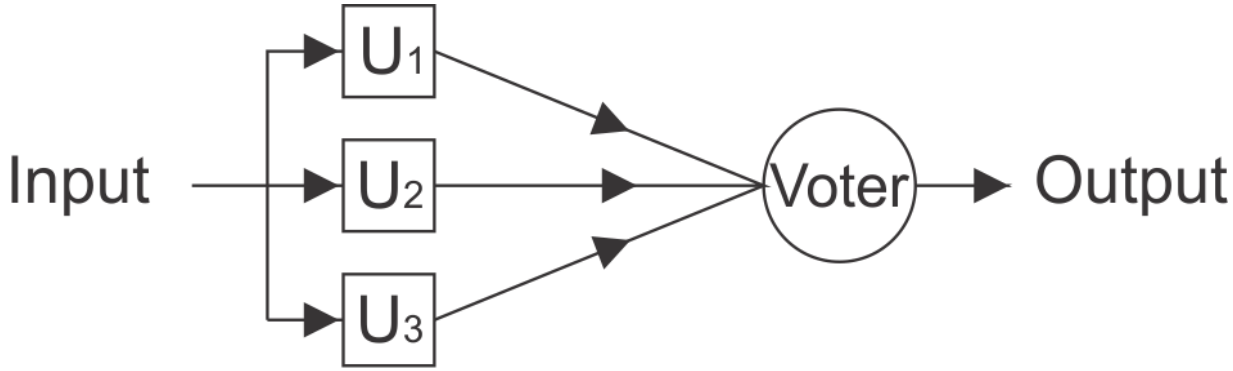
Check Yourself 1

- Calculate the Reliability of TMR system (Triple Modular Redundant system)
  - *Reliability of U1 = U2 = U3 = RU, Voter = RV*



Check Yourself 1 - Solution

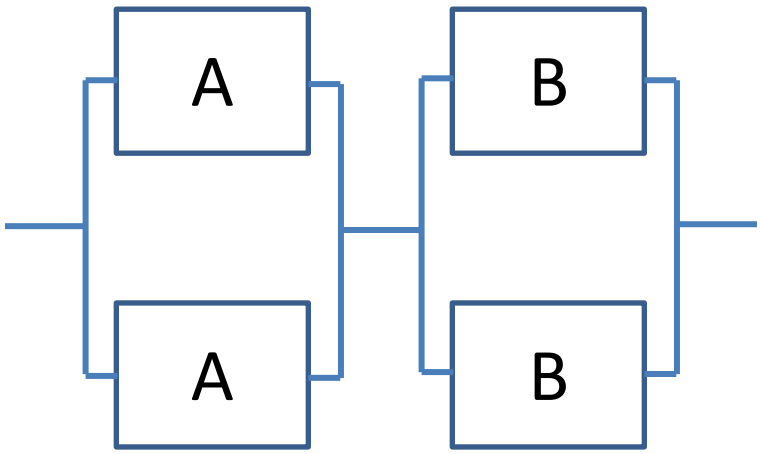
- Calculate the Reliability of TMR system (Triple Modular Redundant system)
  - *Reliability of U1 = U2 = U3 = RU, Voter = RV*



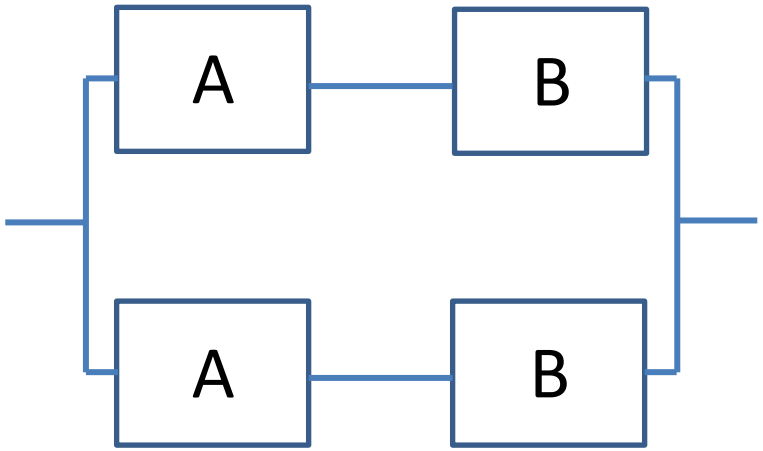
$$R_{tmr} = \left(RU^3 + 3RU^2(1 - RU)\right)RV = (3RU^2 - 2RU^3)RV$$

Check Yourself 2

- Parallel-to-series interconnection scheme and Series-to-parallel interconnection scheme
- Find the both reliabilities, R_{PS} and R_{SP}
- If each subsystem reliability is R, find R_{PS} and R_{SP} and which one is more reliable?



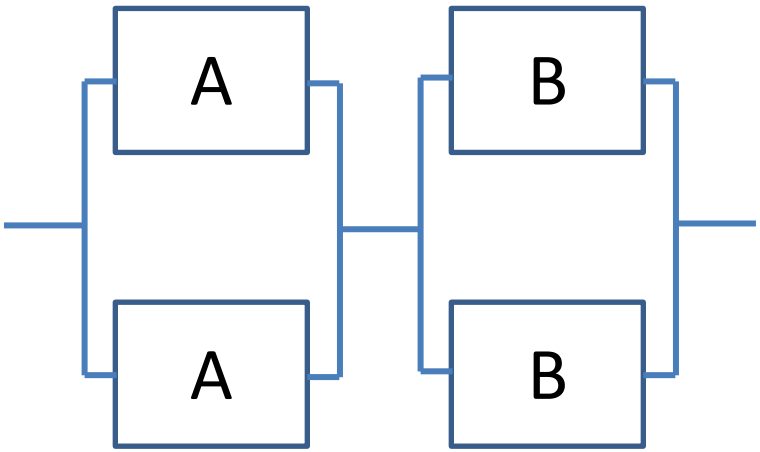
PS scheme



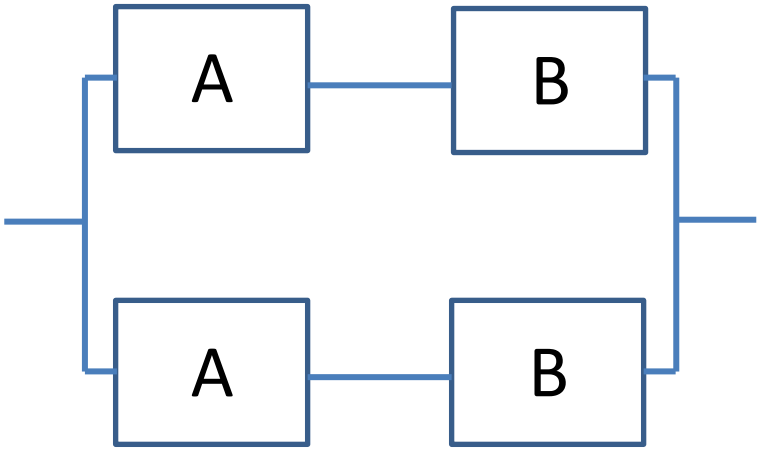
SP scheme

Check Yourself 2 - Solution

- Parallel-to-series interconnection scheme and Series-to-parallel interconnection scheme
- Find the both reliabilities, R_{PS} and R_{SP}
- If each subsystem reliability is R, find R_{PS} and R_{SP} and which one is more reliable?



PS scheme



SP scheme

RPS = [1 – (1 – RA)^2] [1 – (1 – RB)^2] = R^4 – 4 R^3 + 4 R^2

RSP = [1 – (1 – RA RB) (1 – RA RB)] = 2 R^2 – R^4

RPS > RSP

Highly Dependable Computer Systems

Long-life systems: Fail-slow, Rugged, High-reliability
Spacecraft with multiyear missions, systems in inaccessible locations
Methods: Replication (spares), error coding, monitoring, shielding

Safety-critical systems: Fail-safe, Sound, High-integrity
Flight control computers, nuclear-plant shutdown, medical monitoring
Methods: Replication with voting, time redundancy, design diversity

Non-stop systems: Fail-soft, Robust, High-availability
Telephone switching centers, transaction processing, e-commerce
Methods: HW/info redundancy, backup schemes, hot-swap, recovery

Strategies to Handle Faults

- **Fault avoidance**

  Techniques aim to **prevent** faults from entering the system during design stage

- **Fault removal**

  Methods attempt to **find** faults within a system **before** it enters service

- **Fault detection**

  Techniques used during service to **detect** faults within the operational system

- **Fault tolerant**

  Techniques designed to **tolerant** faults, i.e. to allow the system operate correctly in the presence of faults.

Actions to identify and remove errors:
- Design reviews
- Testing
- Use certified tools
- Analysis:
  - Hazard analysis
  - Formal methods - proof & refinement

No non-trivial system can be guaranteed free from error

Must have an expectation of failure and make appropriate provision

Redundancy Techniques

- Hardware redundancy
  - Replicate hardware and compare outputs
  - Detects both permanent and temporary faults
  - Little or no performance impact/ Need area and power
- Information redundancy
  - Encode outputs with error detecting or correcting
  - Less hardware/ Complexity in design
- Temporal (time) redundancy
  - Repeating calculations and comparing results
  - For transient faults (cannot detect permanent faults)
  - Less hardware overhead/ Impacts system performance

Architectural approaches

Simplex systems

- highly reliable components

Dual Systems

- twin identical
- twin **dissimilar**
- control + monitor

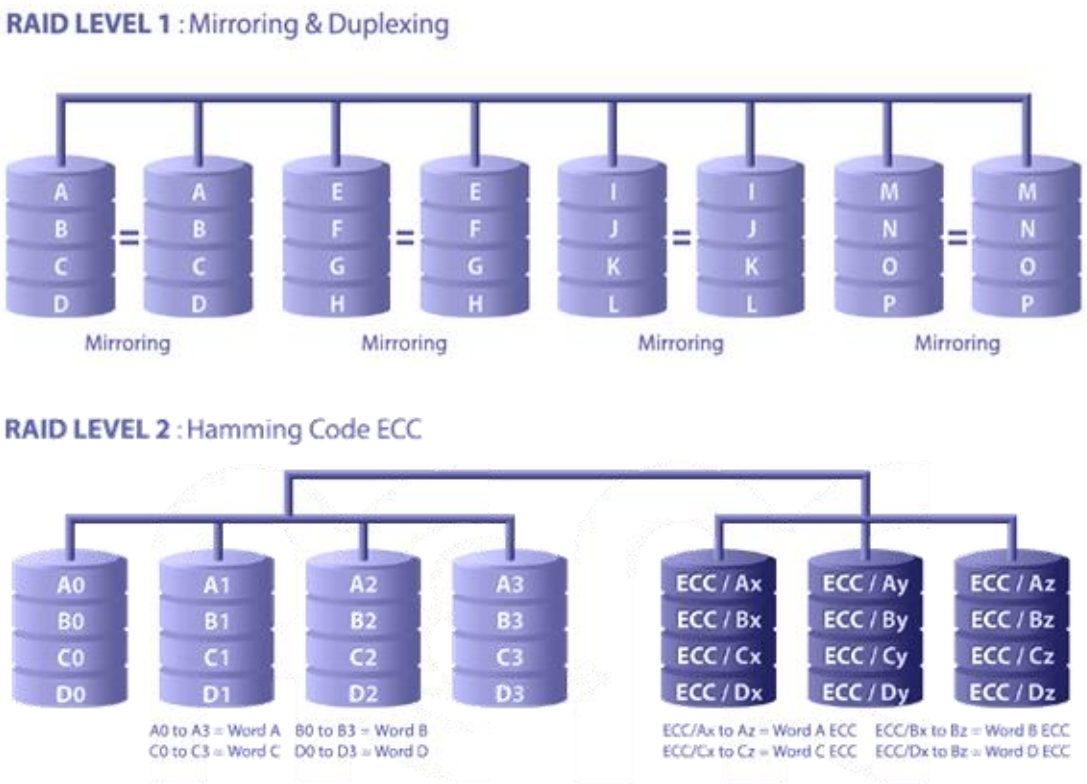
N-way Redundant systems

- identical / dissimilar
- self-checking / voting

Dissimilar systems are also known as "diverse“ systems in which an operation is performed in a different way in the hope that the same fault will not be present in different implementations

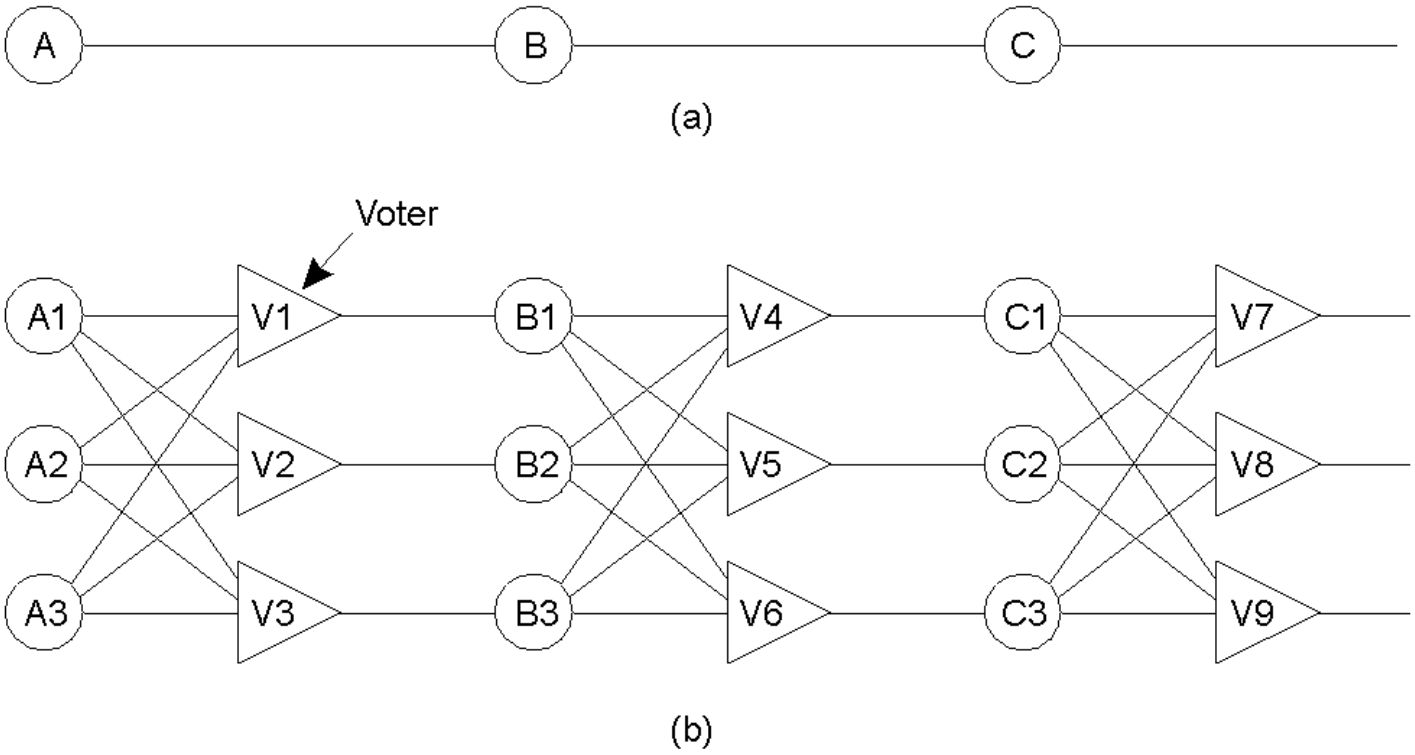
The basic approach to achieve fault tolerance is redundancy

RAID
(Redundant Array of Independent Disks)



RAID has been classified into several levels: 0, 1, 2, 3, 4, 5, 6, 10, 50, each level provides a different degree of fault tolerance

Failure Masking by TMR



(a) Original circuit

(b) Triple modular redundancy (TMR)

Space Shuttle

- Uses **5 identical computers** which can be assigned to redundant operation under program control
- During critical mission phases - boost, re-entry and loading - 4 of its 5 computers operate an **NMR configuration**, receiving the same inputs and executing identical tasks When a failure is detected the computer concerned is switched out of the system leaving a **TMR arrangement**
- The fifth computer is used to perform **non-critical tasks** in a simplex mode, however, under extreme cases may take over critical functions. The unit has "**diverse**" software and could be used if a systematic fault was discovered in the other four computers
- The shuttle can **tolerate up to two computer failures**; after a second failure it operates as a **duplex system** and uses comparison and **self-test techniques** to survive a third fault

Software Faults

❑ Program code (may) contains bugs if actual behavior disagrees with the intended specification. These faults may occur from:

- specification error
- design error
- coding error, e.g. use on un-initialized variables
- integration error
- run time error e.g. operating system stack overflow, divide by zero

❑ Software failure is (usually) deterministic, i.e. predictable, based on the state of the system. There is no random element to the failure – unless the system state cannot be specified precisely. A non-deterministic fault behavior usually indicates that the relevant system state parameters have not been identified.

❑ Fault coverage – defines the fraction of possible faults that can be detected by testing (statement, condition or structural analysis)

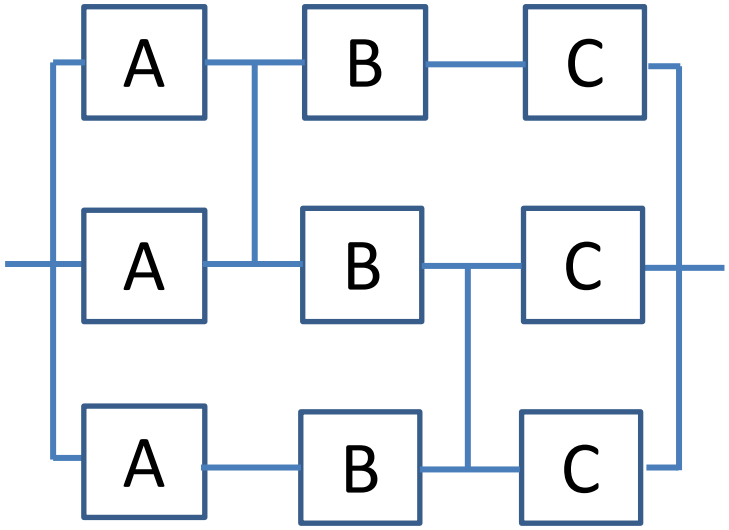
Software Fault Tolerance

N-version programming

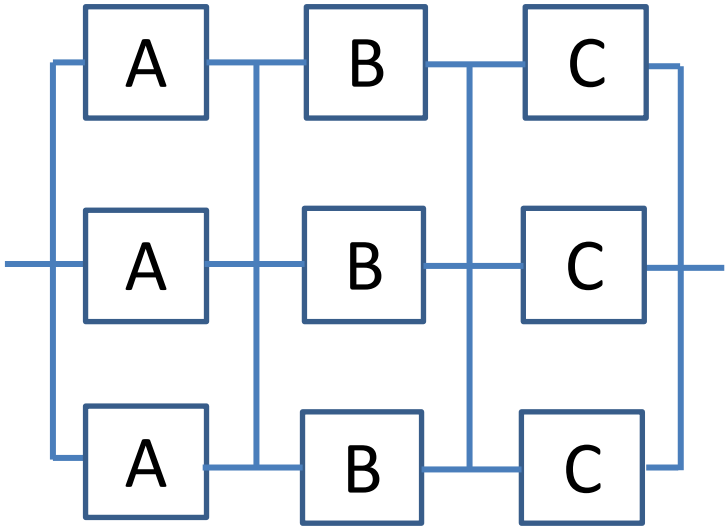
- Use several different implementations of the same specification
- The versions may run *sequentially* on one processor or *in parallel* on different processors.
- They use the same input and their results are compared.
  - In the absence of a disagreement, the result is output.
  - When produced different results:
    - If there are 2 routines:
      - the routines may be repeated in case this was a transient error;
      - to decide which routine is in error.
    - If there are 3 or more routines,
      - voting may be applied to mask the effects of the fault.

HW02: System Reliability

- Find Minimal Path Set (MPS) and Minimal Cut Set (MCS) of each scheme (PP and AP)
- Find the both reliabilities, R_{PP} and R_{AP}
- If each subsystem reliability is R, find R_{PP} and R_{AP} and which one is more reliable?



PP scheme



AP scheme